- **What is the purpose of the `finally` block?**
  It executes **always**, whether an exception occurs or not. It is commonly used for cleanup operations.
- **How does `int.TryParse()` improve robustness compared to `int.Parse()`?**
  `TryParse()` returns `false` instead of throwing an exception when the input is invalid.
- **What exception occurs when accessing `Value` on a null `Nullable<T>`?**
  `InvalidOperationException`.
- **Why is it necessary to check array bounds?**
  To prevent accessing an invalid index and causing an `IndexOutOfRangeException`.
- **How is `GetLength(dimension)` used?**
  It returns the **number of elements in a specific dimension** of a multi-dimensional array.
- **How does memory allocation differ between jagged and rectangular arrays?**
  A rectangular array has a **fixed grid**, while a jagged array consists of **separate arrays**, so each row can have a different size.
- **What is the purpose of nullable reference types?**
  They help identify possible `null` values at **compile time** and reduce null-related errors.
- **What is the performance impact of boxing and unboxing?**
  They add **memory and performance overhead** because boxing creates an object and unboxing requires type conversion.
- **Why must `out` parameters be initialized inside the method?**
  Because the method guarantees that an `out` parameter will receive a value before the method returns.
- **Why must optional parameters appear at the end?**
  Because required parameters must be provided before optional ones, avoiding ambiguity when calling the method.
- **How does the null-propagation operator prevent `NullReferenceException`?**
  It stops the operation and returns `null` if the object before `?.` is `null`.
- **When is a switch expression preferred over a traditional `if` statement?**
  When choosing a value based on **multiple possible patterns or cases**. It is usually more concise.
- **What are the limitations of `params`?**
  - Only **one `params` parameter** is allowed.
  - It must be the **last parameter**.

- It must be a **one-dimensional array** type.